Internships predictions

March 31, 2025

1 Introduction

The objective of this proyect, is to create a predictive model to be used as a recomendation system to predict intern-internship matchs. This model is based on a logistic regression, where we combine data from the internship with data from the intern, so we can use this to predict match suitability for new internships and new interns. As we know, this type of classification gives a probability. This will be used by our employees to accept or discard the pair intern-internship predicted.

2 Data Preparation

As mentioned in the introduction section, the main idea is to combine data from both the internship and the intern to determine whether the pair would be a good match, along with other variables that should provide some predictive value.
The first step was loading and analyzing the selected tables one by one. After this analysis, we dropped the unnecessary columns and imputed the missing data using different methods. Once the data was curated, we merged it to create our working dataframe.
Once our dataframe was merged, we proceeded to identify our features. The first step was to check the numerical correlation with our target variable. Then, we realized that we had to treat these variables as categorical because they represent categories (visa, role, category, destination, etc.).
Since most of our data is categorical, we checked dependencies using hypothesis tests. The test used was the Chi-Squared test to measure dependency between variables.
After obtaining the variables most correlated with our target, we proceeded with feature engineering. The objective here was to combine two columns to create a new one that could provide additional predictive power to our model. In this case, we compared program, working hours, type of internship, destination, and other variables from both the internship and the intern to check whether they were equal or different.

This comparison might provide good predictive value because, for example, having the same destination could make an intern-internship pair more likely to be suitable, whereas different destinations could make the same pair less likely to be a good match
The last step to prepare our data was encoding. We used One-Hot Encoding to encode our categorical data. Additionally, we applied PCA (Principal Component Analysis) with a 95% variance explanation, but this transformation did not bring us better results.

Using this idea, we ended with this variables:
From the internship:

- role
- category
- working hours
- type
- duration
- program.

From the intern:

- primary career
- secondary career
- first location
- second location
- working hours
- accommodation
- internship type
- duration
- destination
- visa

Synthetic columns: These columns show the difference between what the intern wants and what the internship offers

- diff first location
- diff type
- diff language
- diff program

3 Model

Since the goal is to create a recommendation system that outputs percentages, we decided to use a supervised learning model based on logistic regression.

3.1 Baseline Models

Three models were used as a baseline:

- LogisticRegression (Logistic Regression)
- XGBClassifier (XGBoost)
- RandomForestClassifier (Random Forest)
- SVC (SVM)
- KNeighborsClassifier (KNN)
- GradientBoostingClassifier (Gradient Boosting)

After training and testing these models, we select only the top 3 accuracy scores.

- XGBClassifier : 0.7419354838709677
- RandomForestClassifier: 0.7146401985111662
- GradientBoostingClassifier: 0.7295285359801489

3.2 Metrics

As we know, our data is imbalanced, so our metrics will be

- Accuracy
- Recall
- Precision
- Confusion Matrix
- ROC Curve
- AUC Curve

In our case, it is more important to avoid predicting suitable intern-internship pairs as 'not a match' than to predict unsuitable pairs as a 'match.' This is because pairs labeled as a match will undergo human inspection.

3.3 Hyperparameter tuning

Once our models are selected, the next step is to perform hyperparameter tuning. Since our data is imbalanced, we will include some weight-handling techniques in the training process to improve our model's performance. After tuning our models, we obtained the following results:

3.3.1 For XGBoost:

- Accuracy on train: 0.9962732919254659
- Accuracy on test: 0.7518610421836228
- AUC score is 0.700602614157189

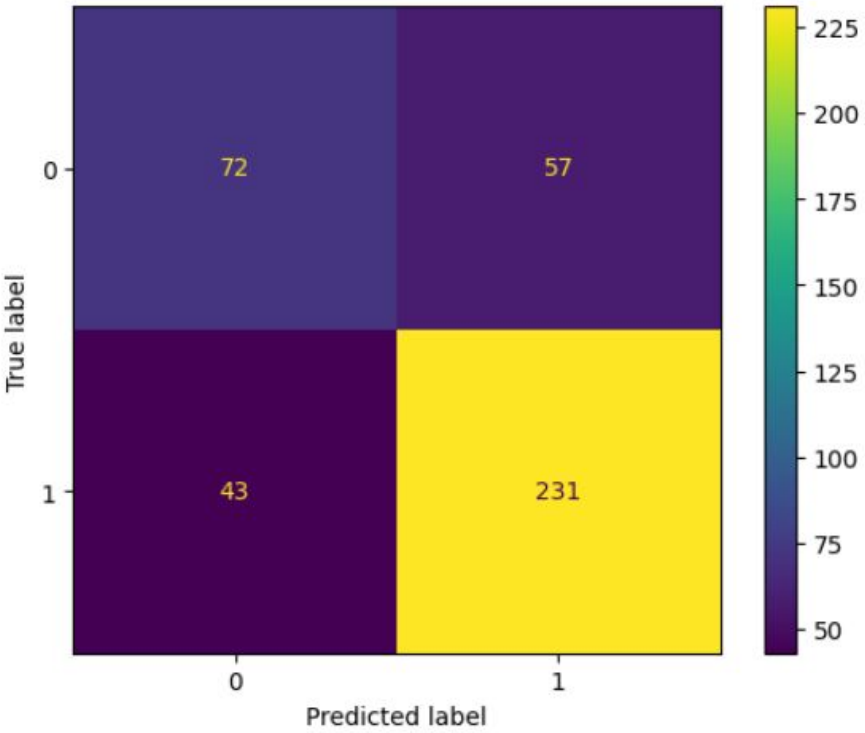

Figure 1: XGBoost Confusion Matrix

	precision	recall	f1-score	support
0	0.63	0.56	0.59	129
1	0.80	0.84	0.82	274
accuracy			0.75	403
macro avg	0.71	0.70	0.71	403
weighted avg	0.75	0.75	0.75	403

Figure 2: XGBoost Classification Report

3.3.2 For Random Forest Classifier:

- Accuracy on train: 0.8664596273291926
- Accuracy on test: 0.7270471464019851
- AUC score is 0.6249221977027104

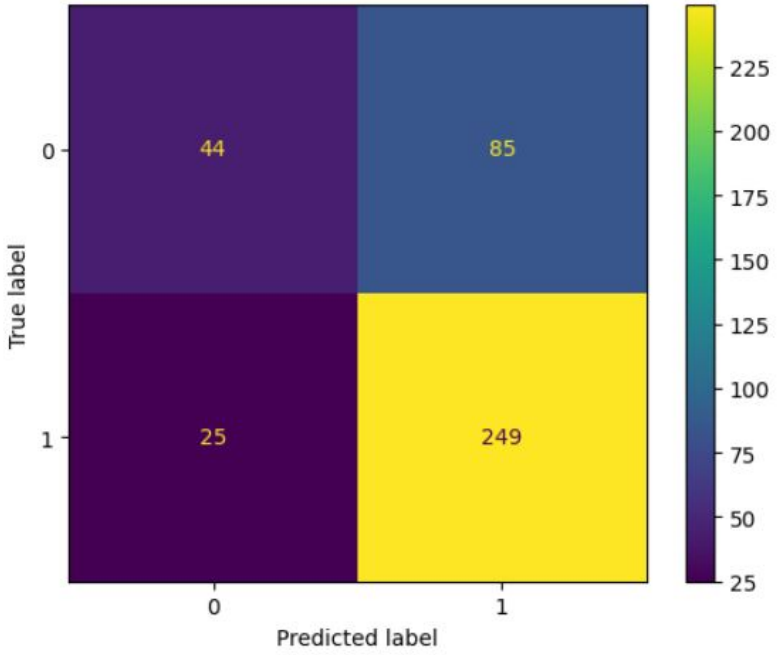

Figure 3: Random Forest Classifier Confusion Matrix

	precision	recall	f1-score	support
0	0.64	0.34	0.44	129
1	0.75	0.91	0.82	274
accuracy			0.73	403
macro avg	0.69	0.62	0.63	403
weighted avg	0.71	0.73	0.70	403

Figure 4: Random Forest Classifier Classification Report

3.3.3 For Gradient Boosting Classifier:

- Accuracy on train: 0.9279503105590062
- Accuracy on test: 0.7295285359801489
- AUC score is 0.6249221977027104

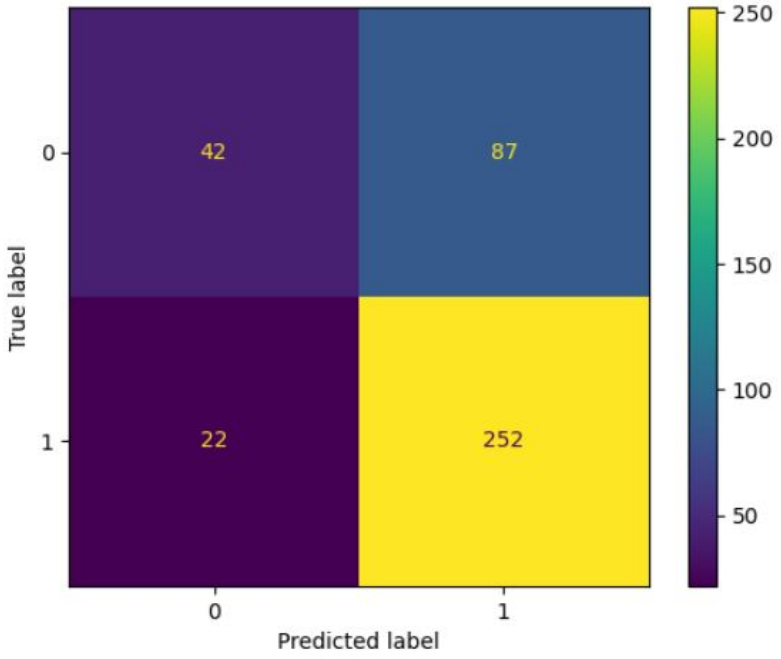

Figure 5: Gradient Boosting Classifie Confusion Matrix

	precision	recall	f1-score	support
0	0.66	0.33	0.44	129
1	0.74	0.92	0.82	274
accuracy			0.73	403
macro avg	0.70	0.62	0.63	403
weighted avg	0.72	0.73	0.70	403

Figure 6: Gradient Boosting Classifier Classification Report

After training and testing these models with the best parameters, we can conclude that XGBoost gives us the best performance.

3.4 Reasons to pick XGBoost model

Random Forest and Gradient Boosting classify the positive class better, meaning that more good matches will be classified correctly. However, most bad matches will be misclassified as good matches. Therefore, we need to put extra effort into reviewing the predictions of these two models.
There is no significant difference between Random Forest and Gradient Boosting. Accuracy, recall, and precision scores are very similar. Recall is a metric that measures how often a machine learning model correctly identifies positive instances—in this case, when a pair is classified as 'not a match' and the correct value is 'not a match'. Precision, on the other hand, measures how well a model predicts positive observations.
XGBoost achieves better recall for the negative class, similar recall for the positive class, higher accuracy compared to the previous models, and a better AUC score. Overall, XGBoost proves to be the best model.

4 Testing

Finally, we tested these three models to check how well they perform with new, unseen data. This data consists of 20 examples, 10 of assigned internships and 10 of unassigned internships
These were the results:

4.0.1 For XGBoost:

- Accuracy: 0.775

4.0.2 For Random Forest Classifier:

- Accuracy: 0.65

4.0.3 For Gradient Boosting Classifier:

- Accuracy on train: 0.65

After this test, we concluded that our model selection was correct.